EJAVA Unit-4

1. Networking Basics [CHP 1 - Networking]

Networking is the process of **connecting two or more computers/devices** to share resources, exchange data, and communicate with each other. In Java, networking is supported through the java.net package.

Advantages:

- Resource sharing
- Data sharing
- Fast communication
- Remote access
- Centralized management

Basic Components of Networking:

- **Node** → Device connected to network
- **Protocol** → Rules for communication
- **IP Address** → Unique address of a device
- **Port Number** → Identifies a specific service
- **Socket** → Endpoint for communication

Common Protocols:

Protocol	Purpose
HTTP	Transfer web pages
HTTPS	Secure web communication
FTP	File transfer
TCP	Reliable communication
UDP	Fast communication
SMTP	Sending emails

Simple Network Flow:
Sender → Network → Receiver

2. Networking Terminologies

Terminology	Description
IP Address	IP address is a unique number assigned to a node of a network (e.g., 192.168.0.1). It is a logical address and can be changed.

Protocol	A protocol is a set of rules followed for communication. Examples: TCP, FTP, Telnet, SMTP, POP.
Port Number	Used to uniquely identify different applications. It acts as a communication endpoint between applications and is associated with an IP address.
MAC Address	MAC (Media Access Control) Address is a unique identifier of NIC (Network Interface Controller). A network node can have multiple NICs, each with a unique MAC address.
Connection-Oriented and Connection-less Protocol	Connection-oriented protocol sends acknowledgement from receiver, so it is reliable but slower (Example: TCP). Connection-less protocol does not send acknowledgement, so it is faster but less reliable (Example: UDP).
Socket	A socket is an endpoint between two-way communication.

3. Java and the Net

The java.net package of J2SE APIs contains a collection of classes and interfaces that provide low-level communication details, allowing programmers to focus on solving problems rather than handling network communication details. It supports common networking protocols like TCP and UDP.

Some classes in java.net package

Class	Description
DatagramPacket	Represents a datagram packet used for sending and receiving data
DatagramSocket	Used for connection-less communication
InetAddress	Represents an IP address
URL	Represents a Uniform Resource Locator
ServerSocket	Used to create server-side sockets
Socket	Used for communication between client and server

Supported Network Protocols

Protocol	Description

TCP (Transmission Control Protocol)	Provides reliable communication between two applications. It is commonly used with Internet Protocol (TCP/IP).
UDP (User Datagram Protocol)	A connection-less protocol that allows transmission of packets between applications.

4. Java InetAddress Class

InetAddress class is used to represent an **IP address** and provides methods to obtain information about hosts and network addresses.

Static Methods of InetAddress Class

Method	Description
public static InetAddress getByName(String host)	Creates an InetAddress object based on the specified hostname
public static InetAddress getByAddress(byte[] addr)	Returns an InetAddress object from a byte array of raw IP address
public static InetAddress[] getAllByName(String host)	Returns an array of InetAddress objects associated with a hostname
public static InetAddress getLocalHost()	Returns the local host address and hostname

Note: These methods throw UnknownHostException during adverse conditions.

Methods of InetAddress Class

Method	Description
public byte[] getAddress()	Returns the raw IP address as an array
public String getHostAddress()	Returns IP address in textual form
public boolean equals(Object obj)	Returns true if the IP address is the same as the specified object

5. Socket Programming

Socket programming is a method of **communication between two nodes on a network**. A socket acts as an **endpoint for two-way communication** between a client and a server.

Features:

- Enables communication between client and server
- Uses TCP or UDP protocols
- Supports two-way communication
- Uses IP address and port number

Types of Sockets

Socket Type	Description
TCP Socket	Connection-oriented and reliable communication
UDP Socket	Connection-less and faster communication

Basic Working of Socket Communication

Client ↔ Socket ↔ Server

Classes used in Socket Programming

Class	Purpose
Socket	Creates client-side socket
ServerSocket	Creates server-side socket
DatagramSocket	Used for UDP communication
DatagramPacket	Represents data packets

Steps in Socket Programming

Step	Description
1	Create server socket
2	Wait for client connection
3	Create client socket
4	Establish connection
5	Send and receive data
6	Close socket

6. TCP/IP Sockets

TCP/IP sockets are used to establish communication between **two applications** over a network using the **Transmission Control Protocol (TCP)**. TCP is a **connection-oriented protocol**, which means a connection must be established before communication begins.

Client Socket

- A client socket initiates a connection request to a server.
- It is created using the Socket class.

Server Socket

- A server socket waits for client requests.
- It is created using the ServerSocket class.

Working of TCP/IP Sockets

1. Server creates a ServerSocket.
2. Server waits for client connection using accept().
3. Client creates a Socket.
4. Connection is established between client and server.
5. Data is transmitted through input and output streams.
6. Communication ends and sockets are closed.

Classes used

- Socket
- ServerSocket

7. TCP/IP Server Sockets

ServerSocket class is used to create a **server application** that waits for requests from clients. It listens on a specific port number and establishes a connection with the client.

Common Constructors:

ServerSocket(int port)
ServerSocket(int port, int backlog)
ServerSocket(int port, int backlog, InetAddress address)

Common Methods:

- accept() → Waits for client connection and returns a socket object
- close() → Closes the server socket
- getInetAddress() → Returns server IP address
- getLocalPort() → Returns local port number
- isClosed() → Checks whether socket is closed

Syntax:
ServerSocket ss = new ServerSocket(5000);
Socket s = ss.accept();

8. Methods of the ServerSocket class

- Socket accept() → Waits for a client connection and returns a socket object
- void bind(SocketAddress host) → Binds the socket to a specific address
- void close() → Closes the socket
- InetAddress getInetAddress() → Returns local IP address information
- int getLocalPort() → Returns the port on which the socket is listening
- SocketAddress getLocalSocketAddress() → Returns local socket address
- boolean isBound() → Checks whether the socket is bound or not
- boolean isClosed() → Checks whether the socket is closed or not

9. TCP/IP Client Sockets

Socket class is used to create **client-side sockets**. A client socket initiates a connection request to the server and establishes communication between client and server.

Constructors of Socket class

Socket(String host, int port)
Socket(InetAddress address, int port)
Socket(String host, int port, InetAddress localAddr, int localPort)

Methods of Socket class

- void connect(SocketAddress host) → Connects the socket to the server
- void close() → Closes the socket
- InetAddress getInetAddress() → Returns IP address of connected server
- InputStream getInputStream() → Returns input stream of socket
- OutputStream getOutputStream() → Returns output stream of socket
- int getPort() → Returns remote port number
- boolean isConnected() → Checks whether socket is connected
- boolean isClosed() → Checks whether socket is closed

10. TCP/IP Sockets

The following steps occur when establishing a TCP connection between two computers using sockets:

1. Create Socket:
The server instantiates a ServerSocket object, denoting which port number communication is to occur on.

2. Identify the Socket:
The server invokes the accept() method of the ServerSocket class. This method waits until a client connects to the server on the given port.

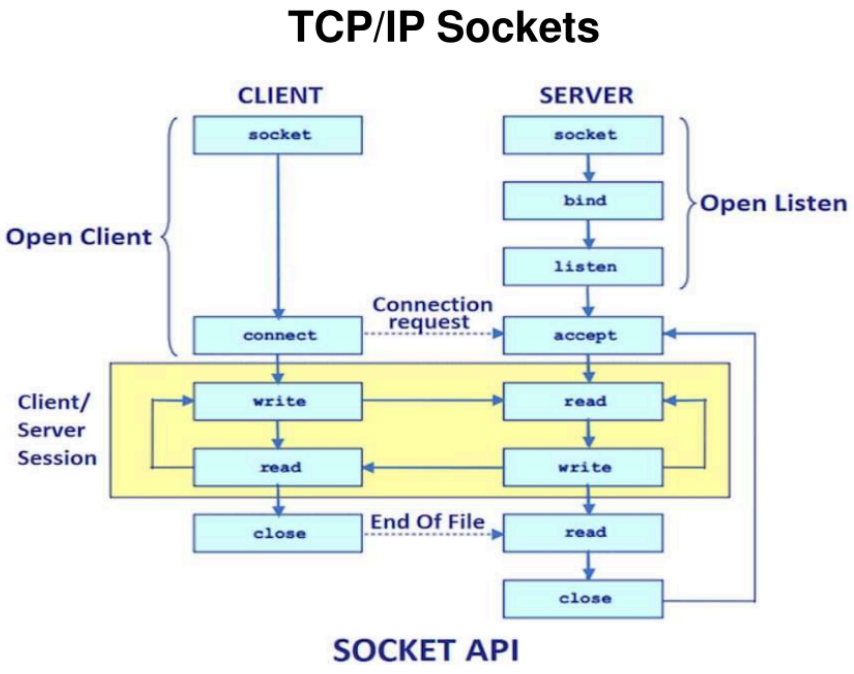
3. On the server, wait for an incoming connection:
After the server is waiting, a client instantiates a Socket object, specifying the server name and the port number to connect to.

4. On the client, connect to server's socket:
The constructor of the Socket class attempts to connect the client to the specified server and port number. If communication is established, the client now has a Socket object capable of communicating with the server.

5. Send and receive Messages:
On the server side, the accept() method returns a reference to a new socket on the server that is connected to the client's socket.

After the connections are established, communication can occur using I/O streams. Each socket has both an OutputStream and an InputStream. The client's OutputStream is connected to the server's InputStream, and the client's InputStream is connected to the server's OutputStream.

6. Close the Socket:
When communication is completed, close the socket using the close() system call.



11. Datagrams

Datagrams are used for **connection-less communication** in Java networking. Datagram communication uses the **User Datagram Protocol (UDP)**, where packets are sent independently without establishing a connection between sender and receiver.

Characteristics:

- Connection-less communication
- Faster than TCP
- No guarantee of delivery
- No acknowledgement mechanism
- Less reliable than TCP

Working of Datagrams

1. Data is divided into packets called datagrams.
2. Each packet contains destination IP address and port number.
3. Packets are transmitted independently.
4. Receiver accepts the packets.

Classes used in Datagram communication

- DatagramPacket
- DatagramSocket

12. Datagrams Packets

DatagramPacket class is used to **send and receive datagram packets**. A datagram packet contains the **data to be transmitted**, along with the **destination IP address and port number**.

Constructors of DatagramPacket

DatagramPacket(byte[] data, int length)

DatagramPacket(byte[] data, int length,
InetAddress ipAddress, int port)

- The first constructor creates a packet for receiving data.
- The second constructor creates a packet for sending data by specifying destination IP address and port number.

13. Datagrams Packet Class

DatagramPacket class represents a **datagram packet**. It is used to **send and receive packets of data** over a connection-less network using UDP. A DatagramPacket contains the data, length of data, IP address, and port number.

Constructors

DatagramPacket(byte[] data, int length)

DatagramPacket(byte[] data, int length,
InetAddress ipAddress, int port)

14. Datagrams Packet Class Methods

- InetAddress getAddress() → Returns the address of the source (for datagrams being received) or destination (for datagrams being sent).
- byte[] getData() → Returns the byte array of data contained in the datagram. Mostly used to retrieve data from the datagram after it has been received.
- int getLength() → Returns the length of the valid data contained in the byte array returned by getData(). This may not equal the length of the whole byte array.
- int getOffset() → Returns the starting index of the data.
- int getPort() → Returns the port number.
- void setAddress(InetAddress ipAddress) → Sets the address to which a packet will be sent. The address is specified by ipAddress.
- void setData(byte[] data) → Sets the data to data, the offset to zero, and the length to number of bytes in data.
- void setData(byte[] data, int idx, int size) → Sets the data to data, the offset to idx, and the length to size.
- void setLength(int size) → Sets the length of the packet to size.
- void setPort(int port) → Sets the port to port.

15. Datagrams Sockets

DatagramSocket class is used for **sending and receiving datagram packets** over a connection-less network using **UDP protocol**. It acts as a communication endpoint for transmitting packets between applications.

Constructors

DatagramSocket()

DatagramSocket(int port)

- DatagramSocket() → Creates a datagram socket and binds it with any available port.
- DatagramSocket(int port) → Creates a datagram socket and binds it with the specified port number.

Methods of DatagramSocket class

- void send(DatagramPacket p) → Sends a datagram packet
- void receive(DatagramPacket p) → Receives a datagram packet
- void close() → Closes the datagram socket
- InetAddress getInetAddress() → Returns the remote IP address
- int getPort() → Returns remote port number
- int getLocalPort() → Returns local port number
- boolean isClosed() → Checks whether the socket is closed or not

16. Java URL

URL (Uniform Resource Locator) is a class in Java used to **represent an address of a resource on the Internet**. It points to resources such as web pages, files, images, or services available on the network.

General Format:

protocol://host:port/file

Components of URL

- **Protocol** → Communication protocol (http, https, ftp)
- **Host** → Domain name or IP address
- **Port** → Port number used by the protocol
- **File** → Resource path or file name

Default Port Numbers

- HTTP → 80
- FTP → 21
- Finger → 79
- HTTPS → 443

Example:

https://www.example.com:443/index.html

- Protocol → https
- Host → www.example.com
- Port → 443
- File → index.html

17. Java URL Class

The URL class is used to **represent a Uniform Resource Locator (URL)**. It provides methods to access information about a URL such as **protocol, host name, port number, file name, and reference**.

Common Constructors

URL(String url)

URL(String protocol,
String host,
int port,
String file)

Methods of URL Class

- String getProtocol() → Returns the protocol name of the URL
- String getHost() → Returns the host name of the URL
- int getPort() → Returns the port number of the URL
- String getFile() → Returns the file name of the URL
- String getPath() → Returns the path of the URL
- String getRef() → Returns the reference (anchor) of the URL
- String toString() → Returns string representation of URL
- URLConnection openConnection() → Opens a connection with the URL

18. RMI (Remote Method Invocation) [CHP 2 - RMI]

RMI (Remote Method Invocation) is a mechanism that allows an **object running in one JVM to invoke methods on an object running in another JVM**. It enables distributed applications where methods of remote objects can be called from different machines.

Components of RMI

- Remote Object
- Stub
- Skeleton
- RMI Registry

Advantages

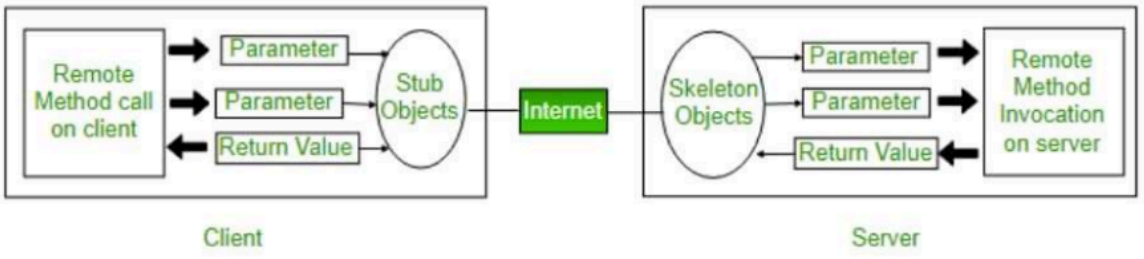
- Supports distributed applications
- Allows communication between different JVMs
- Provides object-to-object communication
- Simplifies remote method calling

Basic RMI Process

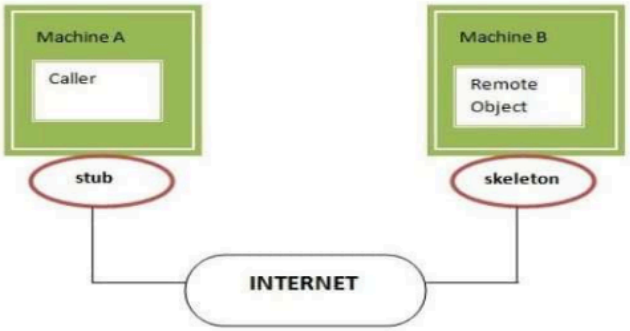
1. Create remote interface
2. Implement the remote interface
3. Create and start RMI Registry
4. Register remote object
5. Client looks up object in registry
6. Client invokes remote methods

19. Working of RMI

The working of RMI involves communication between the **client**, **RMI Registry**, and **remote object**.



Working of RMI



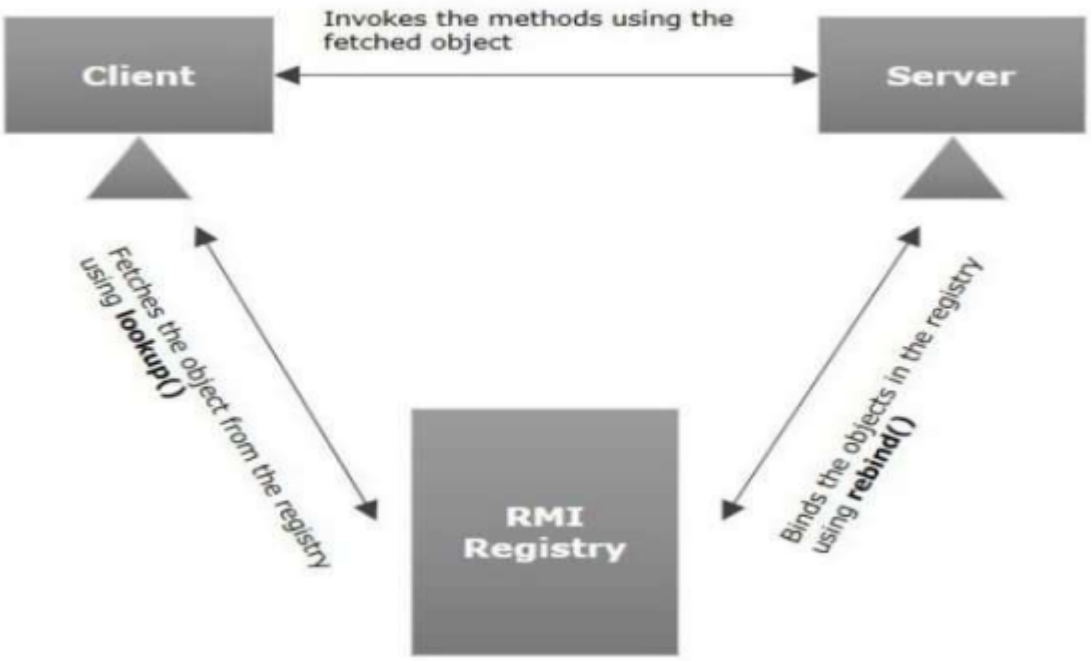
Steps involved:

1. The server creates a **remote object** and registers it with the **RMI Registry**.
2. The **RMI Registry** stores the reference of the remote object.

3. The client searches for the remote object in the **RMI Registry**.
4. The registry returns the reference (stub) of the remote object to the client.
5. The client invokes methods on the remote object through the stub.
6. The request is sent to the server, the method is executed, and the result is returned to the client.

20. RMI Registry

RMI Registry is a **naming service** provided by Java RMI that allows remote objects to be **registered and located by name**. It acts as a central directory where the server registers remote objects and clients look them up.



Functions of RMI Registry

- Stores references of remote objects
- Maps object names with remote objects
- Allows clients to locate remote objects
- Provides communication between client and server

Common Methods

bind()

rebind()

lookup()

unbind()

- bind() → Binds a remote object with a name
- rebind() → Replaces an existing binding
- lookup() → Retrieves remote object reference
- unbind() → Removes a registered object

21. Steps to write RMI program

1. Create the **remote interface** and declare remote methods.
2. Create the **implementation class** for the remote interface.
3. Compile the implementation class and create **stub and skeleton** files.
4. Start the **RMI Registry**.
5. Create the **server application** and register the remote object with the RMI Registry.
6. Create the **client application**.
7. Client searches the remote object using the **RMI Registry**.
8. Invoke methods on the remote object and receive the result.

22. Example of RMI Program

1. Remote Interface

```
import java.rmi.*;

public interface AddInterface extends Remote{
  public int add(int x,int y) throws RemoteException;
}
```

2. Implementation Class

```
import java.rmi.*;
import java.rmi.server.*;

public class AddImpl extends UnicastRemoteObject
implements AddInterface{

  AddImpl() throws RemoteException{
    super();
  }

  public int add(int x,int y){
```

```
    return x+y;
  }
}
```

3. Server Program

```
import java.rmi.*;

public class Server{
  public static void main(String args[]){
    try{
      AddImpl obj = new AddImpl();

      Naming.rebind("rmi://localhost/Add",obj);

      System.out.println("Server Ready");
    }
    catch(Exception e){
      System.out.println(e);
    }
  }
}
```

4. Client Program

```
import java.rmi.*;

public class Client{
  public static void main(String args[]){
    try{
      AddInterface obj =
      (AddInterface)Naming.lookup(
      "rmi://localhost/Add");

      System.out.println(
      "Sum = "+obj.add(10,20));
    }
    catch(Exception e){
      System.out.println(e);
    }
  }
}
```

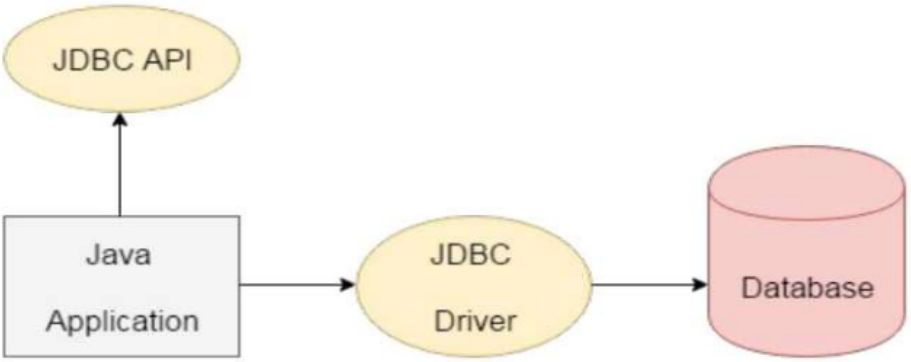
Output

Sum = 30

This follows the RMI flow: **Remote Interface → Implementation Class → Server → Client**.

23. Java JDBC [CHP 3 - JDBC]

JDBC (**Java Database Connectivity**) is an API used to **connect and execute queries with a database**. It provides methods and interfaces for connecting Java applications to databases and retrieving data from the database.



Advantages of JDBC

- Connects Java applications with databases
- Executes SQL queries
- Retrieves and updates data
- Supports multiple database systems

Main JDBC Components

- DriverManager → Manages database drivers
- Connection → Establishes connection with database
- Statement → Executes SQL statements
- ResultSet → Stores retrieved data

Basic JDBC Process

1. Load the JDBC driver
2. Establish database connection
3. Create statement object
4. Execute SQL query
5. Process result

6. Close connection

24. JDBC Drivers

JDBC Driver is a software component that enables a Java application to **interact with a database**. It converts JDBC calls into database-specific calls.

Types of JDBC Drivers

1. Type 1 Driver – JDBC-ODBC Bridge Driver

- Uses ODBC driver to connect to database
- Translates JDBC calls into ODBC calls
- Not platform independent

2. Type 2 Driver – Native API Driver

- Uses native database libraries
- Converts JDBC calls into database-specific API calls
- Requires native library on client system

3. Type 3 Driver – Network Protocol Driver

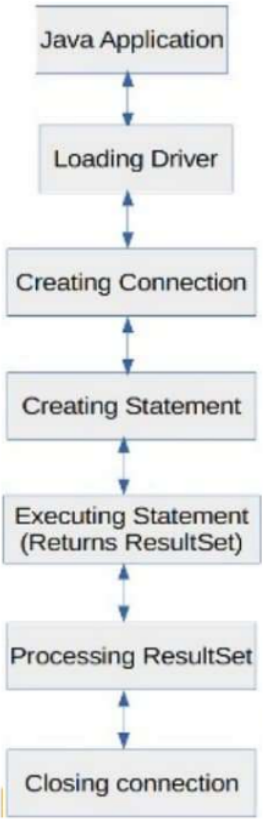
- Uses middleware server
- Converts JDBC calls into network protocol calls
- Database independent

4. Type 4 Driver – Thin Driver

- Converts JDBC calls directly into database protocol
- Pure Java driver
- Platform independent and widely used

25. JDBC Process

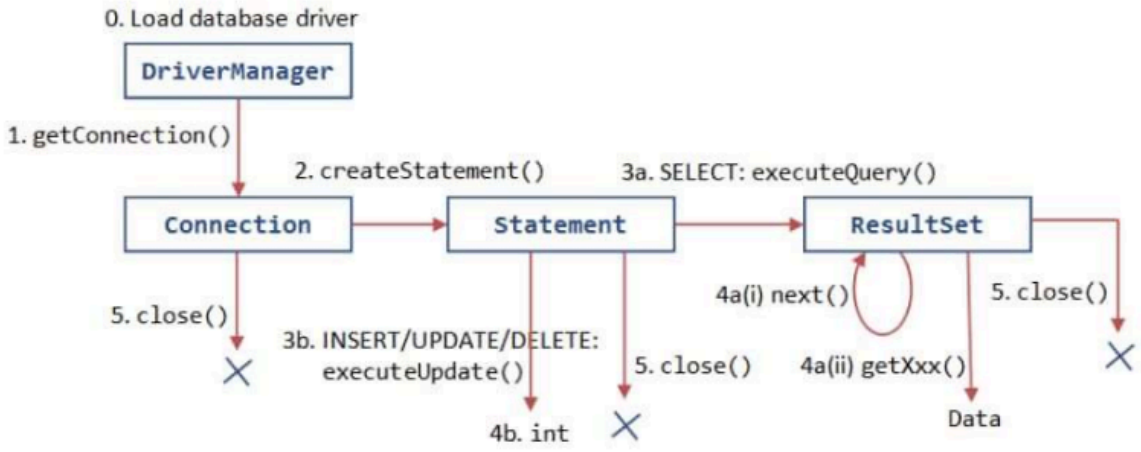
JDBC process is a sequence of steps used to **connect a Java application with a database and perform database operations**.



Steps in JDBC Process

1. **Load the Driver**
   Load the JDBC driver class.
2. **Establish Connection**
   Create a connection between Java application and database.
3. **Create Statement Object**
   Create a statement object to send SQL queries.

4. **Execute Query**
   Execute SQL statements using the statement object.
5. **Process Result**
   Retrieve and process data from the ResultSet.
6. **Close Connection**
   Close database connection and release resources.



26. JDBC Process – DriverManager

DriverManager is a class in JDBC used to **manage database drivers** and establish a connection between the Java application and the database.

Functions of DriverManager

- Loads database drivers
- Establishes connection with database
- Maintains communication between Java application and database

Common Methods

- static Connection getConnection(String url)
  → Establishes connection using URL
- static Connection getConnection(String url, String user, String password)
  → Establishes connection using URL, username, and password
- static void registerDriver(Driver d)
  → Registers a driver with DriverManager
- static void deregisterDriver(Driver d)
  → Removes a registered driver

Example:

```
Class.forName("com.mysql.jdbc.Driver");

Connection con=
DriverManager.getConnection(
"jdbc:mysql://localhost:3306/test",
"root",
"password");
```

27. JDBC Process – Connection Object

Connection is an interface in JDBC used to **establish a session between a Java application and a database**. It provides methods for creating statements and managing database transactions.

Common Methods

- Statement createStatement()
  → Creates a Statement object
- PreparedStatement prepareStatement(String sql)
  → Creates a PreparedStatement object
- CallableStatement prepareCall(String sql)
  → Creates a CallableStatement object
- void close()
  → Closes the database connection
- boolean isClosed()
  → Checks whether the connection is closed or not
- void commit()
  → Saves changes permanently
- void rollback()
  → Undoes changes made during transaction

28. JDBC Process – Statement Object

Statement is an interface in JDBC used to **execute SQL queries and update the database**. It sends SQL statements to the database and retrieves results.

Common Methods

- ResultSet executeQuery(String sql)
  → Executes SELECT query and returns ResultSet
- int executeUpdate(String sql)
  → Executes INSERT, UPDATE, DELETE queries and returns number of affected rows

- boolean execute(String sql)
  → Executes SQL statement and returns true if ResultSet is returned
- void close()
  → Closes the statement object
- int getMaxRows()
  → Returns maximum number of rows
- void setMaxRows(int max)
  → Sets maximum number of rows

Example:

```
Statement stmt = con.createStatement();

ResultSet rs =
stmt.executeQuery(
"SELECT * FROM Student");
```

29. Statement Methods

- ResultSet executeQuery(String sql)
  → Executes SQL SELECT query and returns data in a ResultSet.
- int executeUpdate(String sql)
  → Executes INSERT, UPDATE, and DELETE statements and returns number of affected rows.
- boolean execute(String sql)
  → Executes SQL statement and returns true if the result is a ResultSet.
- void addBatch(String sql)
  → Adds SQL statements into a batch.
- int[] executeBatch()
  → Executes all SQL statements in the batch.
- void close()
  → Closes the statement object.
- void setMaxRows(int max)
  → Sets the maximum number of rows.
- int getMaxRows()
  → Returns the maximum number of rows.

30. Prepared Statement

PreparedStatement is an interface that extends the Statement interface. It is used to **execute parameterized SQL queries**. It compiles SQL queries only once and can execute them multiple times efficiently.

Advantages of PreparedStatement

- Improves performance
- Supports parameterized queries
- Reduces SQL injection problems
- Can execute queries multiple times

Syntax:

```
PreparedStatement ps=
con.prepareStatement(
"insert into Student values(?,?)");
```

Example:

```
PreparedStatement ps=
con.prepareStatement(
"insert into Student values(?,?)");

ps.setInt(1,101);
ps.setString(2,"John");

ps.executeUpdate();
```

31. Prepared Statement Methods

- void setInt(int parameterIndex, int value)
  → Sets an integer value at the specified parameter index.
- void setString(int parameterIndex, String value)
  → Sets a string value at the specified parameter index.
- void setFloat(int parameterIndex, float value)
  → Sets a float value.
- void setDouble(int parameterIndex, double value)
  → Sets a double value.
- void setDate(int parameterIndex, Date value)
  → Sets a date value.
- ResultSet executeQuery()
  → Executes the SQL query and returns a ResultSet.
- int executeUpdate()
  → Executes INSERT, UPDATE, or DELETE statements and returns number of affected rows.
- boolean execute()
  → Executes SQL statement and returns true if the result is a ResultSet.

- void clearParameters()
  → Clears current parameter values.

32. Callable Statement

CallableStatement is an interface that extends PreparedStatement. It is used to **call stored procedures** in a database. It provides methods to set input parameters and retrieve output values from stored procedures.

Advantages of CallableStatement

- Used to execute stored procedures
- Supports input and output parameters
- Improves performance
- Reduces network traffic

Syntax:

```
CallableStatement cs =
con.prepareCall("{call procedureName(?,?)}");
```

Example:

```
CallableStatement cs =
con.prepareCall("{call addData(?,?)}");

cs.setInt(1,101);
cs.setString(2,"John");

cs.execute();
```

33. Stored Procedure

A **Stored Procedure** is a collection of **precompiled SQL statements** stored in the database and executed as a single unit. It is used to perform specific tasks repeatedly and can be called using CallableStatement.

Advantages of Stored Procedure

- Improves performance because SQL statements are precompiled
- Reduces network traffic
- Supports code reusability
- Provides better security

Syntax:

```
CREATE PROCEDURE procedure_name
AS
BEGIN
  SQL statements;
END
```

Example:

```
CREATE PROCEDURE addStudent
(
  @id INT,
  @name VARCHAR(20)
)
AS
BEGIN
  INSERT INTO Student
  VALUES(@id,@name)
END
```

34. Callable Statement Methods

- void registerOutParameter(int parameterIndex, int sqlType)
  → Registers the OUT parameter of the stored procedure.
- void setInt(int parameterIndex, int value)
  → Sets an integer value for the specified parameter.
- void setString(int parameterIndex, String value)
  → Sets a string value for the specified parameter.
- int getInt(int parameterIndex)
  → Retrieves integer value from the OUT parameter.
- String getString(int parameterIndex)
  → Retrieves string value from the OUT parameter.
- boolean execute()
  → Executes the stored procedure.
- ResultSet executeQuery()
  → Executes query and returns a ResultSet.
- int executeUpdate()
  → Executes update statements and returns number of affected rows.

35. JDBC Process – ResultSet Object

ResultSet is an interface in JDBC used to **store and retrieve data returned by SQL queries**. It maintains a cursor that points to the current row of data.

Functions of ResultSet

- Stores records retrieved from database
- Retrieves values from rows and columns
- Allows navigation through records

Common Methods

- boolean next()
  → Moves cursor to the next row
- String getString(int columnIndex)
  → Retrieves string value from specified column
- int getInt(int columnIndex)
  → Retrieves integer value from specified column
- float getFloat(int columnIndex)
  → Retrieves float value from specified column
- double getDouble(int columnIndex)
  → Retrieves double value from specified column
- boolean previous()
  → Moves cursor to previous row
- void close()
  → Closes the ResultSet object

36. Scrollable and Updatable ResultSet

A **Scrollable ResultSet** allows the cursor to move **forward and backward** through records. An **Updatable ResultSet** allows updating data in the database through the ResultSet object.

Scrollable ResultSet

- Cursor can move in both directions
- Can move to any row in the result set
- Supports random access of records

Updatable ResultSet

- Allows modification of records
- Supports insert, update, and delete operations
- Changes can be reflected in the database

Syntax:

```
Statement stmt =
con.createStatement(
ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_UPDATABLE);
```

37. Scrollable ResultSet

A **Scrollable ResultSet** allows the cursor to move **forward and backward through rows** of a result set instead of moving only in the forward direction.

Types of Scrollable ResultSet

- ResultSet.TYPE_FORWARD_ONLY
  → Cursor moves only in forward direction
- ResultSet.TYPE_SCROLL_INSENSITIVE
  → Cursor can move in both directions and does not reflect database changes
- ResultSet.TYPE_SCROLL_SENSITIVE
  → Cursor can move in both directions and reflects database changes

Common Methods

- boolean first()
  → Moves cursor to first row
- boolean last()
  → Moves cursor to last row
- boolean next()
  → Moves cursor to next row
- boolean previous()
  → Moves cursor to previous row
- boolean absolute(int row)
  → Moves cursor to specified row number
- boolean relative(int rows)
  → Moves cursor relative to current position

38. Updatable ResultSet

An **Updatable ResultSet** allows data in the ResultSet object to be **modified and updated directly in the database**. It supports **inserting, updating, and deleting records**.

Features

- Allows modification of database records
- Supports insert, update, and delete operations
- Changes are reflected in the database

Common Methods

- void updateString(int columnIndex, String value)
  → Updates string value in specified column
- void updateInt(int columnIndex, int value)
  → Updates integer value in specified column
- void updateRow()
  → Updates current row in the database
- void deleteRow()
  → Deletes current row
- void insertRow()
  → Inserts a new row
- void moveToInsertRow()
  → Moves cursor to insert row

Syntax:

```
Statement stmt =
con.createStatement(
ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_UPDATABLE);
```

39. JDBC Process – Accessing Data in ResultSet

Data stored in a ResultSet object can be accessed using **getter methods**. The cursor points to rows, and values can be retrieved by specifying the **column index** or **column name**.

Common Methods

- getString(int columnIndex)
  → Returns string value from the specified column
- getInt(int columnIndex)
  → Returns integer value from the specified column
- getFloat(int columnIndex)
  → Returns float value from the specified column
- getDouble(int columnIndex)
  → Returns double value from the specified column
- getString(String columnName)
  → Returns string value from the specified column name

- getInt(String columnName)
  → Returns integer value from the specified column name

Example:

```
ResultSet rs =
stmt.executeQuery(
"select * from Student");

while(rs.next()){
  System.out.println(
  rs.getInt(1)+" "+
  rs.getString(2));
}
```

40. JavaBean [CHP 4 - Java Beans]

JavaBean is a **reusable software component** written in Java that follows certain conventions. It is used to **encapsulate multiple objects into a single object**, allowing data to be accessed and modified through methods.

Characteristics of JavaBean

- Class must implement Serializable interface
- Class should have a **public no-argument constructor**
- Properties should be accessed using **getter and setter methods**
- All fields should be **private**

Syntax:

```
public class StudentBean
implements Serializable{

  private int id;
  private String name;

  public StudentBean(){}

  public int getId(){
    return id;
  }

  public void setId(int id){
    this.id=id;
  }
```

```
  public String getName(){
    return name;
  }

  public void setName(String name){
    this.name=name;
  }
}
```

41. JavaBean – Setter and Getter Methods in Java

Getter and Setter methods are used in JavaBean to **access and modify private data members** of a class. These methods follow a naming convention.

- **Getter Method** → Used to retrieve the value of a property. Method name starts with get or is (for boolean type).
- **Setter Method** → Used to set or modify the value of a property. Method name starts with set.

Syntax:

```
public class StudentBean {

  private int id;
  private String name;

  // Setter methods
  public void setId(int id){
    this.id = id;
  }

  public void setName(String name){
    this.name = name;
  }

  // Getter methods
  public int getId(){
    return id;
  }

  public String getName(){
    return name;
  }
```

}

Example

- setId(101) → Sets value of id
- getId() → Returns value of id

42. Example of JavaBean Class

```
public class StudentBean {

   private int id;
   private String name;

   // No-argument constructor
   public StudentBean(){}

   // Setter methods
   public void setId(int id){
      this.id = id;
   }

   public void setName(String name){
      this.name = name;
   }

   // Getter methods
   public int getId(){
      return id;
   }

   public String getName(){
      return name;
   }

   public static void main(String args[]) {

      StudentBean s = new StudentBean();

      s.setId(101);
      s.setName("John");

      System.out.println("ID : " + s.getId());
```

```
        System.out.println("Name : " + s.getName());
    }
}
```

Output:
ID : 101
Name : John

43. JavaBean Properties

JavaBean properties are **attributes of a JavaBean class** that represent the characteristics of an object. These properties are accessed using **getter and setter methods**.

Types of JavaBean Properties

1. **Simple Property**
    - Represents a single value
    - Uses get() and set() methods

Example:

```
private String name;

public String getName(){
   return name;
}

public void setName(String name){
   this.name = name;
}
```

2. **Indexed Property**
    - Represents an array of values
    - Allows access using index values

Example:

```
private String subjects[];

public String getSubjects(int index){
   return subjects[index];
}

public void setSubjects(int index,
                String subject){
   subjects[index]=subject;
```

}

3. **Bound Property**
    - Generates notifications when property value changes
4. **Constrained Property**
    - Allows changes only after approval from other objects

44. Advantages and Disadvantages of JavaBean

Advantages of JavaBean

- Reusable component that can be used in different applications
- Provides easy maintenance and code reusability
- Supports encapsulation by hiding data members
- Easy to configure and use
- Supports persistence through serialization

Disadvantages of JavaBean

- Requires getter and setter methods, increasing code size
- Can become complex for large applications
- Performance may be reduced due to multiple method calls
- Requires a no-argument constructor

45. JavaBean Introspection

JavaBean Introspection is the process of **analyzing a JavaBean class to identify its properties, methods, and events**. It allows builder tools to discover information about a bean automatically without requiring explicit coding.

Features

- Identifies properties of JavaBean
- Identifies methods of JavaBean
- Identifies events supported by JavaBean
- Helps builder tools use bean information automatically

Classes used for Introspection

- Introspector → Provides methods to analyze a bean
- BeanInfo → Contains information about bean properties, methods, and events
- PropertyDescriptor → Describes bean properties

Example:

BeanInfo info =

Introspector.getBeanInfo(
StudentBean.class);

46. JavaBean JAR File

A **JAR (Java Archive) file** is used to **package JavaBeans and related files into a single compressed file**. It helps in distributing and reusing JavaBeans in different applications.

Advantages of JAR file

- Combines multiple files into a single file
- Reduces file size through compression
- Easy distribution of JavaBeans
- Improves portability and reusability

Steps to create a JavaBean JAR file

1. Compile the JavaBean class.
2. Create a manifest file if required.
3. Package the class files into a JAR file using the jar tool.
4. Add the JAR file to the classpath.
5. Use the JavaBean in applications.

Syntax:

jar cvf BeanFile.jar *.class

- c → Creates a new JAR file
- v → Generates verbose output
- f → Specifies JAR file name